

Cours / TP — Tests et logs avec Python

Développement, tests et diagnostic

Contexte

Un programme qui « fonctionne » n'est pas nécessairement un programme fiable.

Lorsqu'un logiciel est développé, il faut pouvoir :

- vérifier automatiquement son comportement ;
- détecter les erreurs ;
- reproduire un problème ;
- comprendre pourquoi une erreur s'est produite ;
- disposer d'informations utiles pour effectuer un diagnostic.

Deux outils sont particulièrement importants :

- les **tests automatisés**, qui vérifient que le programme se comporte comme prévu ;
- les **logs**, qui enregistrent des événements utiles pendant l'exécution.

Objectifs

- distinguer un test d'un affichage avec `print()` ;
- écrire des tests simples avec `pytest` ;
- tester un comportement nominal ;
- tester un cas limite ;
- tester une exception ;
- comprendre un résultat `PASSED` ou `FAILED` ;
- utiliser le module `logging` de Python ;
- distinguer les niveaux `DEBUG`, `INFO`, `WARNING`, `ERROR` et `CRITICAL` ;
- utiliser les logs pour diagnostiquer un problème.

Prérequis

- Python 3 installé ;
- un terminal ;
- un éditeur de code ;
- des notions de base sur les fonctions et les imports Python.

Organisation du projet

Créer le projet suivant :

```
tests-logs-python/  
├──  
└── app/
```

```
├── __init__.py
├── calculator.py
├── sensor.py
├── tests/
│   ├── test_calculator.py
│   └── test_sensor.py
├── requirements.txt
└── README.md
```

Le fichier `app/__init__.py` peut rester vide.

Le fichier `requirements.txt` contient :

```
pytest
```

Préparer le projet

Depuis le dossier `tests-logs-python/`, créer un environnement virtuel :

```
python -m venv .venv
```

L'activer.

Sous Windows :

```
.venv\Scripts\activate
```

Sous macOS/Linux :

```
source .venv/bin/activate
```

Installer les dépendances :

```
python -m pip install -r requirements.txt
```

Vérifier l'installation :

```
python -m pytest --version
```

Tous les tests doivent ensuite être lancés depuis le dossier racine du projet :

```
tests-logs-python/
```

Partie 1 — Pourquoi tester ?

1.1. Une fonction simple

Créer le fichier `app/calculator.py` :

```
def add(a, b):  
    return a + b
```

Cette fonction reçoit deux valeurs et renvoie leur somme.

Vérification manuelle

Depuis le dossier racine, ouvrir Python :

```
python
```

Puis saisir :

```
from app.calculator import add  
print(add(2, 3))
```

Résultat :

```
5
```

Quitter l'interpréteur avec :

```
exit()
```

Le programme semble fonctionner. Cependant, cette vérification est ponctuelle et doit être refaite manuellement.

1.2. Premier test automatisé

Créer le fichier `tests/test_calculator.py` :

```
from app.calculator import add

def test_add():
    assert add(2, 3) == 5
```

Un test avec `pytest` est généralement une fonction dont le nom commence par `test_`. Par défaut, `pytest` recherche notamment les fichiers dont le nom commence par `test_` ou se termine par `_test.py`, puis les fonctions de test dont le nom commence par ``test_`.

Exécuter les tests :

```
python -m pytest # commande recommandée
# OU
pytest
```

Vous devriez obtenir un résultat similaire à :

```
1 passed in 0.01s
```

Pour obtenir une sortie plus courte :

```
python -m pytest -q
# OU
pytest
```

Comprendre le test

Dans cette instruction :

```
assert add(2, 3) == 5
```

- `add` est la fonction testée ;
- `2` et `3` sont les valeurs fournies ;
- `5` est la valeur attendue ;

- `==` compare la valeur réellement obtenue avec la valeur attendue ;
- `assert` vérifie que la comparaison est vraie.

1.3. Test réussi / test échoué

Modifier temporairement le test :

```
def test_add():  
    assert add(2, 3) == 6
```

Exécuter :

```
python -m pytest # commande recommandée  
# OU  
pytest
```

Le test doit échouer. `pytest` affiche généralement une information similaire à :

```
E          assert 5 == 6  
E           +  where 5 = add(2, 3)
```

Le message indique :

- la valeur attendue : `6` ;
- la valeur obtenue : `5` ;
- l'appel concerné : `add(2, 3)` ;
- le fichier et la ligne du test en échec.

`pytest` fournit automatiquement des informations détaillées sur les sous-expressions d'une assertion, ce qui facilite la recherche de l'erreur.

Remettre ensuite le test dans son état initial :

```
def test_add():  
    assert add(2, 3) == 5
```

Relancer les tests et vérifier que le test réussit de nouveau.

Activité de réflexion

Remplacer temporairement le test par :

```
def test_add():  
    result = add(2, 3)  
    expected = 5  
  
    assert result == expected
```

Comparer cette version avec :

```
assert add(2, 3) == 5
```

La première version est parfois plus facile à lire pour un débutant, tandis que la seconde est plus concise.

Partie 2 — Tester plusieurs comportements

2.1. Ajouter une fonction `divide`

Dans `app/calculator.py`, ajouter :

```
def divide(a, b):  
    if b == 0:  
        raise ValueError("Division par zéro")  
  
    return a / b
```

Cette fonction :

- renvoie le résultat de la division si `b` est différent de zéro ;
- lève une exception `ValueError` si `b` vaut zéro.

2.2. Tester le cas nominal

cas nominal : un cas normal où les données sont valides et où aucune erreur n'est attendue.

Dans `tests/test_calculator.py`, ajouter :

```
def test_divide():  
    assert divide(10, 2) == 5
```

Ne pas oublier d'importer `divide` :

```
from app.calculator import add, divide
```

Lancer les tests :

```
python -m pytest
```

2.3. Tester une exception attendue

Une division par zéro ne doit pas être testée avec une comparaison de résultat, puisque la fonction ne renvoie pas de valeur dans ce cas.

Avec `pytest`, utiliser `pytest.raises()` :

```
import pytest

from app.calculator import add, divide

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)
```

Le test réussit si l'appel à `divide(10, 0)` lève bien une `ValueError`. Il échoue si aucune exception n'est levée ou si une exception d'un autre type est levée.

2.4. Tester le message de l'exception

Il est possible de vérifier aussi le message :

```
def test_divide_by_zero():
    with pytest.raises(ValueError, match="Division par zéro"):
        divide(10, 0)
```

Le paramètre `match` vérifie qu'une expression régulière correspond au texte de l'exception. Dans cet exemple simple, le texte recherché est présent dans le message.

- Pour observer précisément l'exception, utiliser :

```
def test_divide_by_zero():
    with pytest.raises(ValueError) as exc_info:
        divide(10, 0)
```

```
assert str(exc_info.value) == "Division par zéro"
```

Cette forme permet d'accéder à l'objet exception capturé.

2.5. Tester plusieurs cas

Un bon ensemble de tests ne vérifie pas uniquement le cas habituel.

Pour `divide(a, b)`, on peut tester :

Situation	Exemple	Résultat attendu
Cas normal	<code>divide(10, 2)</code>	5
Résultat décimal	<code>divide(5, 2)</code>	2.5
Numérateur nul	<code>divide(0, 5)</code>	0
Dénominateur nul	<code>divide(10, 0)</code>	ValueError
Nombres négatifs	<code>divide(-10, 2)</code>	-5

Ajouter au moins deux tests :

```
def test_divide_decimal():
    assert divide(5, 2) == 2.5

def test_divide_with_zero_numerator():
    assert divide(0, 5) == 0
```

Ajouter également, par exemple :

```
def test_divide_negative_number():
    assert divide(-10, 2) == -5
```

Remarque sur les nombres flottants

Pour les nombres flottants, une comparaison directe avec `==` peut parfois être fragile à cause des approximations liées à leur représentation informatique. `pytest.approx()` permet de comparer des valeurs avec une tolérance.

Exemple :

```
def test_float():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

```
def test_divide_float():  
    assert divide(1, 3) == pytest.approx(0.333333333)
```

2.6. Version complète du fichier de tests

À ce stade, `tests/test_calculator.py` peut contenir :

```
import pytest  
  
from app.calculator import add, divide  
  
def test_add():  
    assert add(2, 3) == 5  
  
def test_divide():  
    assert divide(10, 2) == 5  
  
def test_divide_decimal():  
    assert divide(5, 2) == 2.5  
  
def test_divide_with_zero_numerator():  
    assert divide(0, 5) == 0  
  
def test_divide_negative_number():  
    assert divide(-10, 2) == -5  
  
def test_divide_by_zero():  
    with pytest.raises(ValueError, match="Division par zéro"):  
        divide(10, 0)
```

Lancer :

```
python -m pytest -q
```

Partie 3 — Concevoir des tests

3.1. Une fonction de validation

Créer le fichier `app/sensor.py` :

```
def validate_temperature(value):  
    if value < -50:  
        raise ValueError("Température trop basse")  
  
    if value > 100:  
        raise ValueError("Température trop élevée")  
  
    return value
```

La fonction accepte donc les températures comprises entre $-50\text{ }^{\circ}\text{C}$ et $100\text{ }^{\circ}\text{C}$, bornes incluses.

3.2. Déterminer les tests nécessaires

Avant d'écrire le code, compléter le tableau :

Situation	Valeur	Résultat attendu
Température normale	20	valeur acceptée
Limite basse	-50	valeur acceptée
Limite haute	100	valeur acceptée
Trop basse	-51	ValueError
Trop haute	101	ValueError

Question

Pourquoi est-il intéressant de tester les valeurs -50 et 100 ?

Réponse :

Ce sont les frontières du domaine autorisé. Elles permettent de vérifier que les limites sont inclusives et qu'une modification accidentelle de $<$ en $\leq$, ou de $>$ en $\geq$, sera détectée.

3.3. Écrire les tests

Créer `tests/test_sensor.py` :

```
import pytest  
  
from app.sensor import validate_temperature
```

Ajouter le test du cas normal :

```
def test_temperature_normale():  
    assert validate_temperature(20) == 20
```

Ajouter les tests des limites :

```
def test_temperature_limite_basse():  
    assert validate_temperature(-50) == -50  
  
def test_temperature_limite_haute():  
    assert validate_temperature(100) == 100
```

Ajouter les tests des valeurs invalides :

```
def test_temperature_trop_basse():  
    with pytest.raises(ValueError, match="Température trop basse"):  
        validate_temperature(-51)  
  
def test_temperature_trop_haute():  
    with pytest.raises(ValueError, match="Température trop élevée"):  
        validate_temperature(101)
```

Lancer tous les tests :

```
python -m pytest -q
```

3.4. Faire volontairement une erreur

Modifier temporairement la fonction :

```
def validate_temperature(value):  
    if value < -50:  
        raise ValueError("Température trop basse")  
  
    if value > 50:  
        raise ValueError("Température trop élevée")  
  
    return value
```

Lancer :

```
python -m pytest
```

Le test de la limite haute échoue :

```
def test_temperature_limite_haute():  
    assert validate_temperature(100) == 100
```

En effet, avec `value > 50`, l'appel `validate_temperature(100)` lève une `ValueError`, alors que le test attend que `100` soit accepté.

En revanche, le test :

```
def test_temperature_trop_haute():  
    with pytest.raises(ValueError, match="Température trop élevée"):  
        validate_temperature(101)
```

peut encore réussir, car `101 > 50` est également vrai.

Cela montre l'intérêt de tester **à la fois les valeurs limites et les valeurs situées au-delà des limites**.

Remettre ensuite la condition correcte :

```
if value > 100:  
    raise ValueError("Température trop élevée")
```

Relancer les tests.

Commandes de diagnostic

Exécuter tous les tests :

```
python -m pytest
```

Exécuter uniquement les tests du calculateur :

```
python -m pytest tests/test_calculator.py
```

Exécuter uniquement les tests du capteur :

```
python -m pytest tests/test_sensor.py
```

Afficher davantage de détails :

```
python -m pytest -v
```

Exécuter un seul test :

```
python -m pytest tests/test_sensor.py::test_temperature_limite_haute
```

Partie 4 — Les logs

4.1. Pourquoi utiliser des logs ?

Un programme peut effectuer de nombreuses opérations sans afficher quoi que ce soit.

Lorsqu'un problème apparaît, il peut alors être difficile de savoir :

- ce que le programme était en train de faire ;
- quelle étape a échoué ;
- quelle valeur a été reçue ;
- dans quel ordre les opérations se sont déroulées.

Les **logs** permettent de conserver des informations sur l'exécution du programme.

Un log ne remplace pas un test. Un test vérifie un comportement attendu ; un log conserve une trace de ce qui s'est passé pendant une exécution réelle.

Exemple

```
assert add(2, 3) == 5
```

Cette instruction vérifie une règle.

```
logger.info("Connexion au capteur établie")
```

Cette instruction signale qu'un événement s'est produit.

4.2. Premier exemple avec **logging**

Créer le fichier `app/logger_demo.py` :

```
import logging

logger = logging.getLogger(__name__)

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s - %(message)s",
)

logger.debug("Message DEBUG")
logger.info("Programme démarré")
logger.warning("Température élevée")
logger.error("Impossible de lire le capteur")
logger.critical("Arrêt du système")
```

Depuis le dossier racine du projet, exécuter :

```
python app/logger_demo.py
```

Résultat attendu :

```
INFO - Programme démarré
WARNING - Température élevée
ERROR - Impossible de lire le capteur
CRITICAL - Arrêt du système
```

Le message `DEBUG` n'apparaît pas, car le seuil est fixé à `INFO`.

Décomposer l'exemple

```
logger = logging.getLogger(__name__)
```

Crée un logger associé au module courant.

```
logging.basicConfig(...)
```

Configure le système de journalisation.

```
level=logging.INFO
```

Indique que les messages de niveau **INFO** et plus grave sont pris en compte par la configuration.

```
format="%(levelname)s - %(message)s"
```

Indique la forme des messages affichés.

Pour les logs comportant des valeurs variables, il est recommandé de transmettre les valeurs comme arguments :

```
temperature = 78.4  
  
logger.info("Température mesurée : %.1f °C", temperature)
```

Cette forme laisse au module **logging** le soin de construire le message uniquement lorsqu'il doit être produit.

Remarque : par défaut, les messages du module **logging** sont envoyés vers la sortie d'erreur standard (**stderr**), tandis que **print()** écrit normalement sur la sortie standard (**stdout**). Dans un terminal, les deux apparaissent généralement à l'écran.

4.3. Les niveaux de journalisation

Les niveaux standards sont classés du moins grave au plus grave :

Niveau	Utilisation
DEBUG	Informations détaillées utiles au diagnostic
INFO	Fonctionnement normal du programme
WARNING	Situation inhabituelle, mais le programme peut continuer
ERROR	Une opération n'a pas pu être réalisée
CRITICAL	Problème très grave pouvant empêcher le programme de continuer

Exemples

```
logger.debug("Valeur brute reçue : %s", raw_value)  
logger.info("Connexion au capteur établie")  
logger.warning("Température proche de la limite")  
logger.error("Lecture du capteur impossible")  
logger.critical("Service indisponible")
```

Un niveau de log ne provoque pas automatiquement une exception ou l'arrêt du programme. Il indique la gravité que le développeur attribue à l'événement.

4.4. Modifier le niveau de journalisation

Modifier :

```
level=logging.INFO
```

en :

```
level=logging.DEBUG
```

Relancer :

```
python app/logger_demo.py
```

Question

Quelle différence observez-vous ?

Réponse :

Avec le niveau **DEBUG**, les messages **DEBUG** et les messages plus graves apparaissent. Avec le niveau **INFO**, les messages **DEBUG** sont ignorés.

On peut résumer la règle ainsi :

Seuil configuré	Messages visibles
DEBUG	DEBUG, INFO, WARNING, ERROR, CRITICAL
INFO	INFO, WARNING, ERROR, CRITICAL
WARNING	WARNING, ERROR, CRITICAL
ERROR	ERROR, CRITICAL
CRITICAL	CRITICAL

4.5. Écrire des logs utiles

Un log doit apporter une information exploitable.

Bon exemple :

```
temperature = 78.4

logger.info("Température mesurée : %.1f °C", temperature)
```

Exemples trop vagues :

```
logger.info("Ça marche")
logger.error("Erreur")
```

Un bon log doit aider à répondre à plusieurs questions :

- que s'est-il passé ?
- dans quelle partie du programme ?
- avec quelle valeur ?
- à quel moment ?
- l'opération a-t-elle réussi ou échoué ?

Pour cela, un format avec l'heure et le nom du module peut être utile :

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
)
```

Exemple de sortie :

```
2026-09-29 09:00:00,123 - __main__ - INFO - Programme démarré
```

Les formats de `logging` peuvent notamment inclure l'heure, le nom du logger, le niveau et le message.

Partie 5 — Logs et capteur

5.1. Préparer le module `sensor`

Créer `app/sensor.py` et coller ce code :

```
import logging

logger = logging.getLogger(__name__)
```

```
def validate_temperature(value):
    if value < -50:
        raise ValueError("Température trop basse")

    if value > 100:
        raise ValueError("Température trop élevée")

    return value

def read_temperature(value):
    logger.info("Lecture de la température")

    if value < -50 or value > 100:
        logger.error(
            "Température invalide : %.1f °C",
            value,
        )
        raise ValueError("Température hors limites")

    if value > 80:
        logger.warning(
            "Température élevée : %.1f °C",
            value,
        )

    logger.info(
        "Température valide : %.1f °C",
        value,
    )

    return value
```

Déroulement de la fonction

Pour une valeur valide et inférieure ou égale à 80 :

1. un log **INFO** indique le début de la lecture ;
2. un log **INFO** indique que la température est valide ;
3. la valeur est renvoyée.

Pour une valeur strictement supérieure à 80 et inférieure ou égale à 100 :

1. un log **INFO** indique le début de la lecture ;
2. un log **WARNING** signale une température élevée ;
3. un log **INFO** indique que la valeur reste valide ;
4. la valeur est renvoyée.

Pour une valeur hors de l'intervalle [-50, 100] :

1. un log **INFO** indique le début de la lecture ;

2. un log `ERROR` signale la valeur invalide ;
3. une `ValueError` est levée ;
4. la fonction ne renvoie pas de valeur.

5.2. Créer le programme principal

Créer `app/main.py` :

```
import logging

from app.sensor import read_temperature

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s - %(message)s",
)

temperature = read_temperature(85)

print(f"Température : {temperature} °C")
```

Depuis le dossier racine du projet, exécuter :

```
python app/main.py
```

Résultat attendu, à quelques détails de format près :

```
INFO - Lecture de la température
WARNING - Température élevée : 85.0 °C
INFO - Température valide : 85.0 °C
Température : 85 °C
```

5.3. Tester plusieurs situations

Dans `app/main.py`, tester successivement :

```
temperature = read_temperature(20)
```

puis :

```
temperature = read_temperature(85)
```

puis :

```
temperature = read_temperature(120)
```

Pour tester la dernière situation sans faire terminer le programme avec une trace d'erreur, utiliser temporairement :

```
try:
    temperature = read_temperature(120)
    print(f"Température : {temperature} °C")
except ValueError as error:
    print(f"Lecture impossible : {error}")
```

Résultats attendus

Valeur	Logs attendus	Programme
20	INFO pour le début et la validation	continue
85	INFO, WARNING, puis INFO	continue
120	INFO, puis ERROR	l'exception est levée

Questions

Pour chaque cas :

1. Quel niveau de log apparaît ?
2. Le programme continue-t-il ?
3. Une exception est-elle générée ?

Partie 6 — Diagnostic à partir des logs

6.1. Lire une séquence de logs

Vous recevez les logs suivants :

```
INFO - Démarrage du système
INFO - Connexion au capteur
INFO - Lecture de la température
WARNING - Température élevée : 82.3 °C
```

```
INFO - Lecture de la température  
ERROR - Température invalide : 145.2 °C  
CRITICAL - Arrêt du système
```

Questions

1. Le système a-t-il démarré correctement ?
2. La première température est-elle valide ?
3. Pourquoi la première valeur produit-elle un **WARNING** et non un **ERROR** ?
4. Quelle valeur pose problème ?
5. Quelle opération échoue ?
6. Quel événement semble provoquer l'arrêt ?
7. Le message **CRITICAL** arrête-t-il le programme à lui seul ?
8. Quelle information supplémentaire serait utile ?

Réponses

- Le système a démarré et s'est connecté au capteur.
- La température **82.3 °C** est élevée, mais elle reste dans les limites acceptées.
- La température **145.2 °C** est hors limites.
- La lecture de cette seconde valeur échoue.
- Le système s'arrête ensuite, mais l'arrêt doit être provoqué par une exception ou par une instruction explicite, pas par le niveau **CRITICAL** lui-même.
- Des informations supplémentaires pourraient être :
 - la date et l'heure ;
 - le nom du capteur ;
 - l'identifiant de la machine ;
 - la fonction concernée ;
 - la trace de l'exception ;
 - la valeur limite configurée.

6.2. Ecrire dans un fichier

Les logs peuvent être envoyés dans un fichier :

```
logging.basicConfig(  
    filename="application.log",  
    filemode="w",  
    level=logging.INFO,  
    format="%(asctime)s - %(levelname)s - %(message)s",  
    encoding="utf-8",  
)
```

- **filemode="w"** remplace le contenu à chaque exécution ;
- le mode **"a"** ajoute les nouveaux messages à la fin ;
- **encoding="utf-8"** permet de gérer correctement les caractères accentués sur les versions modernes de Python.

Lancer ensuite :

```
python app/main.py
```

Observer le fichier :

```
application.log
```

- Sous macOS/Linux : `cat application.log`
- Sous PowerShell : `Get-Content application.log`
- Sous Windows CMD : `type application.log`

Partie 7 — Tester les logs

7.1. Tester qu'un avertissement est généré

Dans `tests/test_sensor.py`, ajouter :

```
import logging

import pytest

from app.sensor import read_temperature, validate_temperature
```

Puis :

```
def test_temperature_elevee_genere_un_warning(caplog):
    with caplog.at_level(logging.INFO):
        result = read_temperature(85)

    assert result == 85
    assert "Température élevée" in caplog.text
```

`caplog.at_level(logging.INFO)` demande à `pytest` de capturer les messages de niveau `INFO` et plus grave pendant le bloc `with`.

La fixture `caplog` de `pytest` permet de capturer les logs produits pendant un test et de vérifier leur contenu.

7.2. Tester le niveau du log

Le test précédent vérifie que le message apparaît. On peut aller plus loin et vérifier précisément le niveau du log.

Pour vérifier précisément le niveau :

```
def test_temperature_elevee_genere_un_warning(caplog):
    with caplog.at_level(logging.INFO):
        read_temperature(85)

    assert any(
        record.levelno == logging.WARNING
        and "Température élevée" in record.message
        for record in caplog.records
    )
```

Ce test vérifie simultanément :

- qu'un message a été produit ;
- qu'il est de niveau `WARNING` ;
- qu'il contient l'information attendue.

7.3. Tester le log d'erreur

```
def test_temperature_invalide_genere_une_erreur(caplog):
    with caplog.at_level(logging.INFO):
        with pytest.raises(ValueError, match="Température hors limites"):
            read_temperature(120)

    assert any(
        record.levelno == logging.ERROR
        and "Température invalide" in record.message
        for record in caplog.records
    )
```

Le test vérifie ici deux effets différents d'une même situation : l'exception destinée au programme appelant et le log destiné au diagnostic.

7.4. Tester le log normal (sans erreur ni warning)

```
def test_temperature_normale_ne_genere_pas_de_warning(caplog):
    with caplog.at_level(logging.INFO):
        result = read_temperature(20)

    assert result == 20
    assert not any(
        record.levelno == logging.WARNING
```

```
)  
    for record in caplog.records
```

Un test de logs peut vérifier **la présence** d'un événement, mais aussi l'absence d'un événement indésirable.

7.5. Exécuter les tests des logs

```
python -m pytest tests/test_sensor.py -q
```

Puis :

```
python -m pytest tests/test_sensor.py -v
```

Pour exécuter uniquement le test du **WARNING** :

```
python -m pytest  
tests/test_sensor.py::test_temperature_elevee_genere_un_warning -v
```

Pour exécuter uniquement le test de l'erreur :

```
python -m pytest  
tests/test_sensor.py::test_temperature_invalide_genere_une_erreur -v
```

Comme **caplog capture** les logs, ils ne sont pas nécessairement affichés à l'écran.

Pour afficher les logs en direct :

```
python -m pytest tests/test_sensor.py --log-cli-level=INFO
```

Ou avec un format plus simple :

```
python -m pytest tests/test_sensor.py \  
    --log-cli-level=INFO \  
    --log-cli-format="%(levelname)s - %(message)s"
```

Sous Windows, utiliser plutôt :

```
python -m pytest tests/test_sensor.py --log-cli-level=INFO --log-cli-format="%(levelname)s - %(message)s"
```

Note: `caplog` et `--log-cli-level` ont deux objectifs différents.

- `caplog` permet au **test de vérifier** les logs ;
- `--log-cli-level` permet de **voir les logs pendant l'exécution de pytest**.

Partie 8 — Tests et logs : deux rôles différents

Les tests et les logs sont complémentaires, mais ils ne répondent pas à la même question.

Tests	Logs
Vérifient un comportement attendu	Décrivent l'exécution
Peuvent être automatisés	Sont produits pendant l'exécution
Produisent généralement PASSED ou FAILED	Produisent des événements classés par niveau
Détection des régressions	Aident à comprendre le contexte d'un problème
S'exécutent dans un environnement contrôlé	Peuvent être utilisés en développement et en production
Vérifient des résultats, exceptions ou effets attendus	Conservent des informations de diagnostic

Exemple de test

```
assert add(2, 3) == 5
```

Question :

Le résultat est-il correct ?

Exemple de log

```
logger.info("Connexion au capteur établie")
```

Question :

Que s'est-il passé pendant l'exécution ?

Un log **ERROR** n'est pas toujours synonyme de test échoué, et un test réussi ne signifie pas qu'aucun log d'avertissement n'a été produit. Ces deux informations doivent être analysées séparément.

Partie 9 — Exercice de synthèse

9.1. Ajouter une fonction

Dans `app/sensor.py`, ajouter :

```
def is_temperature_safe(value):  
    ...
```

La fonction doit :

- retourner **True** si la température est comprise entre **0** et **80 °C**, bornes incluses ;
- retourner **False** dans les autres cas.

Une implémentation possible est :

```
def is_temperature_safe(value):  
    logger.info("Vérification de la température : %.1f °C", value)  
  
    if value < 0 or value > 80:  
        logger.warning(  
            "Température non sûre : %.1f °C",  
            value,  
        )  
        return False  
  
    logger.info(  
        "Température acceptable : %.1f °C",  
        value,  
    )  
    return True
```

9.2. Écrire les tests

Créer les tests suivants :

```
def test_temperature_safe_normale():  
    assert is_temperature_safe(20) is True  
  
def test_temperature_safe_limite_basse():  
    assert is_temperature_safe(0) is True
```

```
def test_temperature_safe_limite_haute():
    assert is_temperature_safe(80) is True

def test_temperature_unsafe_trop_basse():
    assert is_temperature_safe(-1) is False

def test_temperature_unsafe_trop_haute():
    assert is_temperature_safe(81) is False
```

Ne pas oublier de modifier l'import :

```
from app.sensor import (
    is_temperature_safe,
    read_temperature,
    validate_temperature,
)
```

L'utilisation de `is True` et `is False` rend explicite que la fonction doit renvoyer un booléen.

9.3. Tester les logs de synthèse

Ajouter :

```
def test_temperature_unsafe_genere_un_warning(caplog):
    with caplog.at_level(logging.INFO):
        result = is_temperature_safe(81)

    assert result is False
    assert any(
        record.levelno == logging.WARNING
        and "Température non sûre" in record.message
        for record in caplog.records
    )
```

Questions

1. Pourquoi utilise-t-on `WARNING` et non `ERROR` pour une température non sûre ?
2. Pourquoi la fonction renvoie-t-elle `False` au lieu de lever une exception ?
3. Dans quelle situation une exception serait-elle plus appropriée ?
4. Quel test vérifie le comportement ?
5. Quel test vérifie la présence du log ?
6. Que se passerait-il si la limite `80` devenait `79` ?

Réponses

- **WARNING** convient si le programme peut continuer malgré la situation.
- **ERROR** ou une exception seraient plus adaptés si l'opération ne peut pas être réalisée.
- Les tests de résultat vérifient la valeur de retour.
- **caplog** vérifie le contenu et le niveau des messages.
- Le test de la limite **80** détecterait une modification incorrecte de la condition.
- Le test de la limite **80** détecterait une modification incorrecte de la condition.

Conclusion

```
TEST
|
|— Vérifie un comportement
|— Compare résultat attendu / obtenu
|— Peut détecter une régression

LOG
|
|— Décrit l'exécution
|— Aide au diagnostic
|— Permet de comprendre un problème
```

Annexe — Commandes utiles

Créer un environnement virtuel :

```
python -m venv .venv
```

Activer l'environnement virtuel sous Linux/macOS :

```
source .venv/bin/activate
```

Activer l'environnement virtuel sous Windows PowerShell :

```
.venv\Scripts\Activate.ps1
```

Sous Windows, si l'invite de commandes classique est utilisée :

```
.venv\Scripts\activate.bat
```

Installer les dépendances :

```
python -m pip install -r requirements.txt
```

Lancer tous les tests :

```
python -m pytest
```

Lancer les tests avec davantage de détails :

```
python -m pytest -v
```

Lancer un fichier de tests :

```
python -m pytest tests/test_calculator.py
```

Lancer un test particulier :

```
python -m pytest tests/test_calculator.py::test_add
```

Afficher les logs capturés pendant les tests :

```
python -m pytest -o log_cli=true --log-cli-level=INFO  
python -m pytest --log-cli-level=INFO
```

Configurer le format des logs affichés par `pytest` :

```
python -m pytest \  
  --log-cli-level=INFO \  
  --log-cli-format="%(levelname)s - %(message)s"  
  
python -m pytest --log-cli-level=INFO --log-cli-format="%(levelname)s - %  
(message)s"
```

Écrire les logs des tests dans un fichier :

```
python -m pytest --log-file=pytest.log --log-file-level=INFO
```
